Leaf motion limits 3D reconstruction of plants in wind: a synthetic benchmark and a study of per-part rigid-motion models

Bibek Gyawali[a,*]

[a] *TODO affiliation,*

Abstract

3D Gaussian splatting and neural radiance fields are now used to reconstruct plants, but they assume that nothing in the scene moves. Leaves move whenever there is wind. We built SynthTree, a generator of synthetic RGB-D videos of potted saplings in which every tree is filmed twice along the same camera path: once in still air and once in wind. Leaves are real geometry and are moved by a simple closed-form wind model, so the true position of every leaf is known in every frame; automatic checks on all 40 sequences confirm that this ground truth matches the images to within 0.007 px. In 108 training runs on four trees, wind at 5 m/s lowers the leaf F-score of 3D Gaussian splatting by 0.341 and of a depth-supervised NeRF by 0.348, while image quality (PSNR) changes by only -7 %: the images look right although the leaves are in the wrong place. Simulated image and depth noise matters much less than wind. Our deformable Gaussian baseline does not help, because its learned motion collapses into one small shift of the whole tree. We then asked why. The collapse remains at 30 frames per second, so the frame rate is not the cause. A per-cell rigid-motion model removes the collapse and reduces the wind part of the chamfer error by 18 %; grouping the same cells by the true leaf identities gives a further small gain, but about three quarters of that error remain, and a second synchronised camera does not help. The learned leaf motion stays far smaller than the true motion. When the same model is given the true leaf motion, the wind error disappears (F-score 0.468 in wind against 0.402 in calm air), so estimating fast leaf motion from images, not representing it, is the main open problem. All results come from simulation.

Keywords: 3D plant reconstruction, Gaussian splatting, neural radiance

*Corresponding author

Email address: TODO-institutional-email@example.org (Bibek Gyawali)

fields, wind, leaf motion, synthetic data, phenotyping

1. Introduction

3D models of plants are used to measure leaf area, leaf angles and canopy shape without harming the plant (Paulus, 2019). Neural radiance fields (Mildenhall et al., 2020; Müller et al., 2022) and 3D Gaussian splatting (3DGS) (Kerbl et al., 2023) can build such models from an ordinary video, and they are quickly being adopted for plants. They all assume a still scene. Real leaves rarely keep still: in a light breeze they flutter several times per second and whole branches sway (de Langre, 2008), while filming a full circle around a sapling takes tens of seconds.

When the 3D model of a real plant is poor, it is hard to say why. Thin leaves that hide each other are difficult for any method, image and depth data are noisy, and moving leaves break the still-scene assumption. On real data these causes cannot be told apart, because nobody knows where every leaf was at every moment. Existing datasets show still scenes (Jensen et al., 2014; Knapitsch et al., 2017; Sturm et al., 2012) or plant point clouds without leaf motion (Schunck et al., 2021).

We therefore built a synthetic benchmark in which every factor is controlled and every leaf is tracked exactly, and used it to study both how much wind hurts reconstruction and what limits methods that try to model the motion. Our contributions are:

1. **SynthTree**, an open, fully scripted generator of procedural saplings, wind and RGB-D video with optional image and depth noise. It produces *paired* sequences: the same tree and the same camera path, once calm and once windy.
2. **Exact per-leaf ground truth in wind**: leaf identity images and the position and orientation of every leaf in every frame, checked to sub-pixel accuracy on all sequences.
3. **An evaluation method** that compares reconstructions with the true leaf surface at each moment and splits the error into a method part, a noise part and a wind part.
4. **A benchmark** showing that wind is the main source of error, that image metrics hide most of it, and that comparing with the still (rest) pose overstates it.
5. **A study of motion models**: a per-cell rigid-motion model removes the collapse of our deformable baseline but recovers little geometry, neither a higher frame rate nor a second camera helps, and the learned

motion is far too small. Given the true leaf motion, the same model reconstructs windy leaves as well as calm ones, so estimating the motion, not representing it, is the bottleneck.

Code and data are available (Section 8).

2. Related work

Plant reconstruction. Laser scanning, multi-view stereo and RGB-D cameras are established tools for measuring plant shape (Paulus, 2019). Pheno4D (Schunck et al., 2021) provides point clouds of growing plants with organ labels, and organ tracking between scans has been studied over hours and days (Li et al., 2013). At those time scales, wind during one scan is treated as noise. The way plants move in wind, from slow branch sway to fast leaf flutter, is well described in biomechanics (de Langre, 2008); wind-tunnel experiments on a whole tree show that individual leaf flutter dominates at low wind speeds and branch buffeting at high speeds (Tadrist et al., 2018), and multi-camera systems have been used to measure how trees deform in wind (Kammel and Reiterer, 2024). For crops photographed from a drone in wind, leaf motion has been removed with optical flow to obtain a still reconstruction (Rochow et al., 2025). How much wind lowers the accuracy of a 3D reconstruction, however, has not been measured. Real datasets are recorded in still air and cannot provide the true shape of moving leaves.

Reconstructing still and moving scenes. NeRF (Mildenhall et al., 2020), its faster variants (Müller et al., 2022), 3DGS (Kerbl et al., 2023) and the surface-based 2DGS (Huang et al., 2024a) reconstruct still scenes; we use the implementations in Nerfstudio (Tancik et al., 2023). Dynamic versions learn how the scene deforms over time (Pumarola et al., 2021; Luiten et al., 2024; Wu et al., 2024; Yang et al., 2024). Some split the scene into parts that move rigidly, such as superpoints (Wan et al., 2024) or sparse control points (Huang et al., 2024b), or share a small set of motion bases between all points (Wang et al., 2024); others give each Gaussian its own trajectory (Lin et al., 2024), and methods aimed at oscillating scenes such as leaves in wind add explicit per-Gaussian motion curves and rigidity terms (Tzathas et al., 2026). Closest to our study, Chen and Lou (2026) model each rigid part of three synthetic trees as a damped oscillator and conclude that reconstructing wind-driven vegetation from one moving camera is severely underconstrained: the learned motion parameters do not match the true ones. With one moving camera, how well motion can be recovered depends on how far the camera moves compared with the scene (Gao et al., 2022).

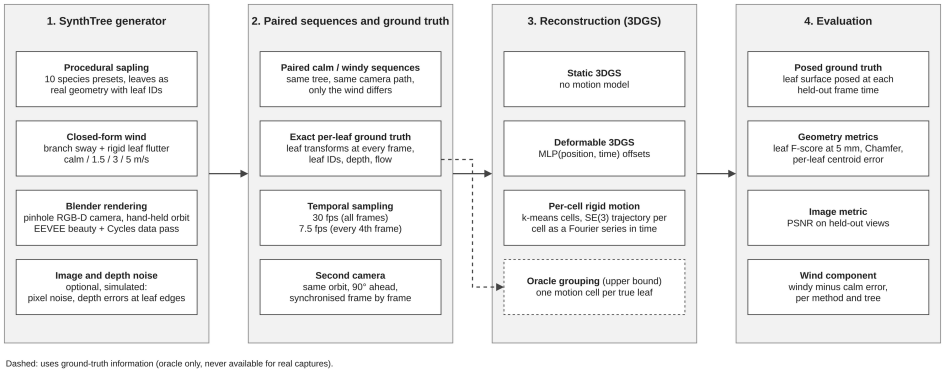


Figure 1: Overview: SynthTree generates a tree, moves it with wind and renders it (1); calm and windy videos come with exact per-leaf ground truth (2); reconstruction methods are trained on them (3) and compared with the true leaf surface (4). Dashed: uses ground truth (oracle only).

Robust variants ignore content that does not fit, either with per-image appearance codes (Martin-Brualla et al., 2021) or by down-weighting pixels with large errors (Sabour et al., 2023); our trimmed loss follows the second idea. None of these methods has been tested on fluttering leaves against ground truth.

Synthetic data. Rendering pipelines such as BlenderProc (Denninger et al., 2023) and ICL-NUIM (Handa et al., 2014) give exact ground truth for rigid indoor scenes, and procedural plant models have a long history (Prusinkiewicz and Lindenmayer, 1990; Weber and Penn, 1995; Runions et al., 2007). Synthetic trees have been used to test whether physical motion parameters can be recovered (Chen and Lou, 2026). As far as we know, no benchmark provides paired still and windy plants with exact per-leaf ground truth for measuring how accurately the leaves themselves are reconstructed in 3D.

3. Generating trees, wind and images

Figure 1 gives an overview. Figure 2 shows the 10 saplings and Fig. 4 what is saved for every frame. Everything is written in Python and NumPy except the rendering, which uses Blender. The same settings and random seed always give the same geometry, camera poses and ground truth, which we checked across two Blender versions.

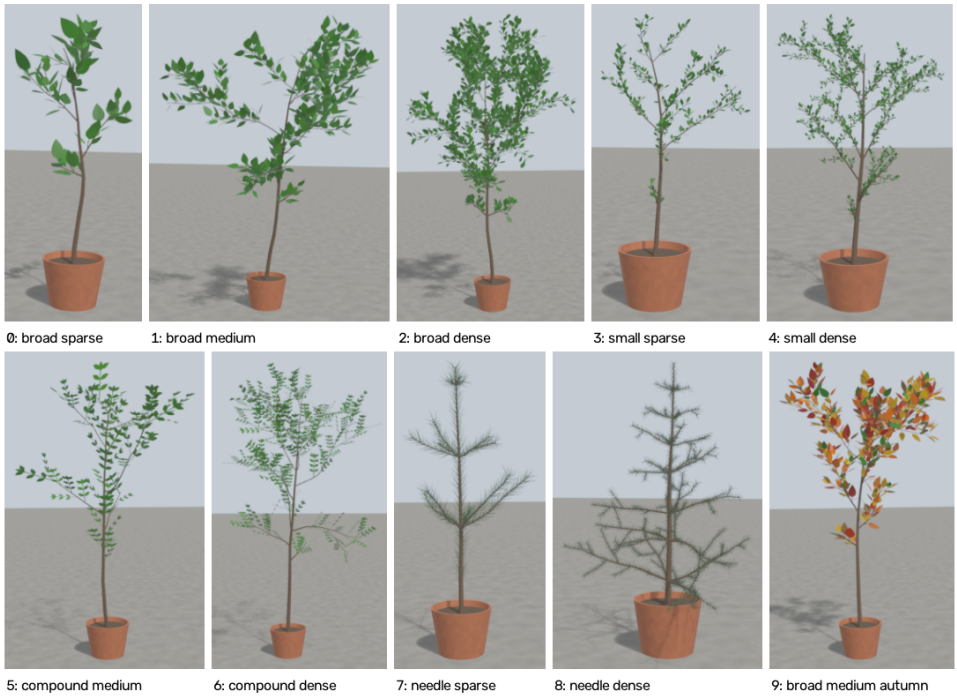


Figure 2: The 10 saplings: broad, small, compound and needle leaves at different densities. Leaves are real 3D meshes.

Table 1: The 10 trees. The experiments use trees 1, 3, 5 and 7 (one per leaf type).

ID	Preset	Leaf type	Height (m)	Leaves	Leaf area (m^2)	Triangles	Split
0	broad_sparse	broad	1.00	86	0.161	2,890	train
1	broad_medium	broad	1.54	624	0.874	17,234	train
2	broad_dense	broad	1.98	2,591	2.460	65,376	train
3	small_sparse	small	0.87	535	0.064	9,190	train
4	small_dense	small	0.82	1,865	0.119	29,992	train
5	compound_medium	compound	1.44	62	0.257	6,378	train
6	compound_dense	compound	1.73	217	0.505	27,044	train
7	needle_sparse	needle	1.14	1,672	0.078	28,107	train
8	needle_dense	needle	1.11	8,192	0.146	136,748	test
9	broad_medium_autumn	broad	1.47	557	0.829	15,366	test

3.1. Trees with leaf identities

Each tree is a skeleton of branches grown with random bends, an upward tendency and regular branching patterns (Weber and Penn, 1995). The wood is a tube mesh around the skeleton. Leaves are real triangle meshes with species-specific outlines, cupping and droop; compound leaves have leaflets on a stalk, and needles are thin prisms. Every leaf has a number, an attachment point, a parent branch, a rest pose, an area and a centre (Table 1). Trees are 0.82 to 1.98 m tall and have 62 to 8,192 leaves.

3.2. Wind

We do not run a physics simulation. Instead, one closed-form function moves the rendered leaves and also produces the exported ground truth, so the two always agree. $\mathrm{Rot}(\mathbf{a}, \theta)$ is the rotation by angle θ about the unit axis $\mathbf{a}$, and Table 2 gives the value of every parameter; all trees use the same values.

Branch sway. Every skeleton node j bends by

$$\Delta_j(t) = \mathrm{Rot}\big(\mathbf{b}, \theta_j(t)\big)\,\mathrm{Rot}\big(\mathbf{w}, \psi_j(t)\big), \tag{1}$$

where $\mathbf{w}$ is the wind direction and $\mathbf{b} = \mathbf{e}_z \times \mathbf{w}$. Bends add up from the root to the tips, $A_j = A_{p(j)}\Delta_j$ and $\mathbf{p}_j = \mathbf{p}_{p(j)} + A_{p(j)}(\mathbf{p}^0_j - \mathbf{p}^0_{p(j)})$, so outer branches move most. The bend angles are

$$\theta_j = c_j\, g_b(U_e)\,[\alpha_s + \alpha_o \sin(2\pi f_B t + \phi_B)], \tag{2}$$

$$\psi_j = c_j\, g_b(U_e)\,\alpha_c \sin(2\pi r f_B t + \phi'_B), \tag{3}$$

with one frequency f_B and phase ϕ_B per branch B, and a flexibility $c_j \propto (r_0/r_j)\,\ell_j$ that makes thin, long segments bend more.

Table 2: Wind model parameters used in the benchmark.

Symbol	Value
U	0 (calm), 1.5, 3 and 5 m/s
U_r	5 m/s (branches and leaves)
$\alpha_s,\ \alpha_o,\ \alpha_c$	6°, 3°, 1.5° (summed along the trunk at $U = U_r$)
c_j	$(r_0/r_j)\,\ell_j \,/\, \sum_{k\in\text{trunk}}(r_0/r_k)\,\ell_k$ r_0: radius at the root; r_j: radius at node j; ℓ_j: length of the segment to the next node
f_B	0.6 Hz (trunk) to 2.0 Hz (finest branches), linear in branch order, times e^{ϵ}, $\epsilon \sim \mathcal{N}(0, 0.12^2)$
r	1.37
$\beta_m,\ \beta_t,\ \beta_p$	15°, 18°, 25° (at $g_l = 1$)
f_ℓ	uniform in $[2.5, 9]$ Hz, per leaf
r_ℓ	uniform in $[1.1, 1.6]$, per leaf
s	1.5
γ	0.35
$n(t)$	$\sum_{k=1}^{5} a_k \sin(2\pi f_k t + \phi_k) / \sum_k a_k$, f_k uniform in $[0.05, 0.6]$ Hz, $a_k = f_k^{-1/2}$; $\lvert n\rvert \le 1$
$\phi_B,\ \phi'_B,\ \phi_\ell,\ \phi'_\ell,\ \phi_k$	uniform in $[0, 2\pi)$, drawn from the tree seed

Leaf flutter. Each leaf rotates as a rigid body about its stalk,

$$\begin{aligned} R_\ell = \operatorname{Rot}(\mathbf{m}_\ell, g_l\beta_m)\operatorname{Rot}\big(\mathbf{x}_\ell, g_l\beta_t \sin(2\pi r_\ell f_\ell t + \phi'_\ell)\big) \\ \operatorname{Rot}\big(\mathbf{y}_\ell, g_l\beta_p \sin(2\pi f_\ell t + \phi_\ell)\big), \end{aligned} \tag{4}$$

with its own frequency f_ℓ between 2.5 and 9 Hz and a mean tilt away from the wind about $\mathbf{m}_\ell = \mathbf{n}_\ell \times \mathbf{w}$. The leaf also follows its branch, $A_\ell = A_{n(\ell)} R_\ell$, and its transform from the rest pose, $\mathbf{x}_t = A_\ell \mathbf{x}_0 + \mathbf{t}_\ell$, is saved for every frame.

Wind strength and gusts. Branch bending grows with the square of the wind speed, $g_b(U) = (U/U_r)^2$, like drag on a flexible body; leaf flutter grows roughly linearly and then levels off, $g_l(U) = s \tanh\big((U/U_r)/s\big)$, in line with wind-tunnel measurements in which flutter depends only weakly on wind speed while branch motion grows strongly (Tadrist et al., 2018). Gusts vary the speed smoothly over time, $U_e(t) = U \max\{0,\, 1 + \gamma\, n(t)\}$. All angles are exactly zero at $U = 0$, so calm videos show the tree exactly in its rest pose. The random phases depend only on the tree, so the wind levels of one tree differ only in strength. We measure leaf motion as the *leaf-centre displacement from the rest pose, median over leaves and frames*, and use this definition throughout. At 5 m/s it is 9 to 60 mm across the 10 trees (15–48 mm for trees 1, 3, 5 and 7), and the median leaf rotation is 23–27°

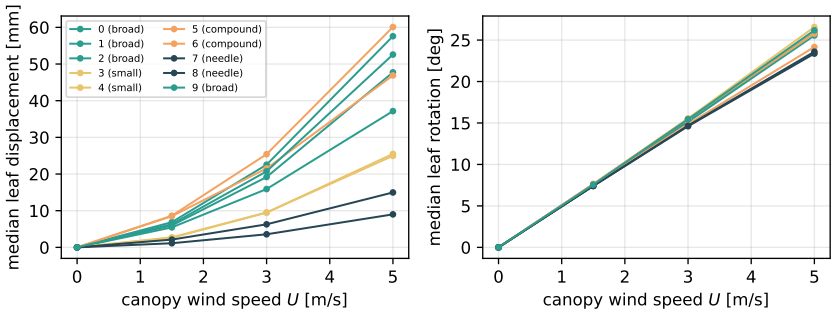


Figure 3: Leaf motion at each wind speed: leaf-centre displacement from the rest pose and leaf rotation, median over leaves and frames.

(Fig. 3). These amplitudes are plausible defaults, not measurements of a particular plant, so we report leaf motion alongside wind speed.

3.3. Rendering and ground truth

The pinhole camera has 848×480 pixels, $f_x = f_y = 603\,\text{px}$ and principal point $(423.5, 239.5)$, a field of view of $70° \times 43°$. Its path is computed from the tree alone. Let h be the height of the axis-aligned bounding box of tree and pot, and w half of its larger horizontal side. The camera looks at the centre of the box and circles it once at radius $R = \max\{1.15\,\max(h/(2\tan 21.7°),\ w/\tan 35.1°) + w,\ 0.6\,\text{m}\}$, so that the whole box fits in the image with a 15 % margin, at a height of $0.1\,h$ above the box centre. For trees 1, 3, 5 and 7 this gives $R = 1.6$–$3.0\,\text{m}$ (about $1.75\,h$) and an elevation of 3°. One circle takes 20 s at 30 fps. The benchmark uses every fourth frame (150 frames, 2.4° apart) and Section 6 also uses all 600. Handheld shaking adds a smooth random offset to the camera position (10 mm RMS per axis) and a rotation in the camera frame (0.6° RMS per axis). Each is a sum of four sinusoids between 0.1 and 1.2 Hz with amplitudes proportional to $f^{-1/2}$, with frequencies and phases drawn from the tree seed. The camera path is therefore identical for all wind levels. Colour images are rendered with Blender's EEVEE or Cycles renderer. Ground truth is rendered in a separate pass with one sample per pixel, so values are never blended across leaf edges. For each pixel we save the 3D position, normal, leaf number, class (leaf, wood, pot, floor) and the position in the next frame, from which we get exact depth and optical flow. Motion blur and rolling shutter can be rendered but are *switched off in all experiments*: every image is a sharp snapshot, which is kinder to the methods than a real handheld recording would be (Section 7).

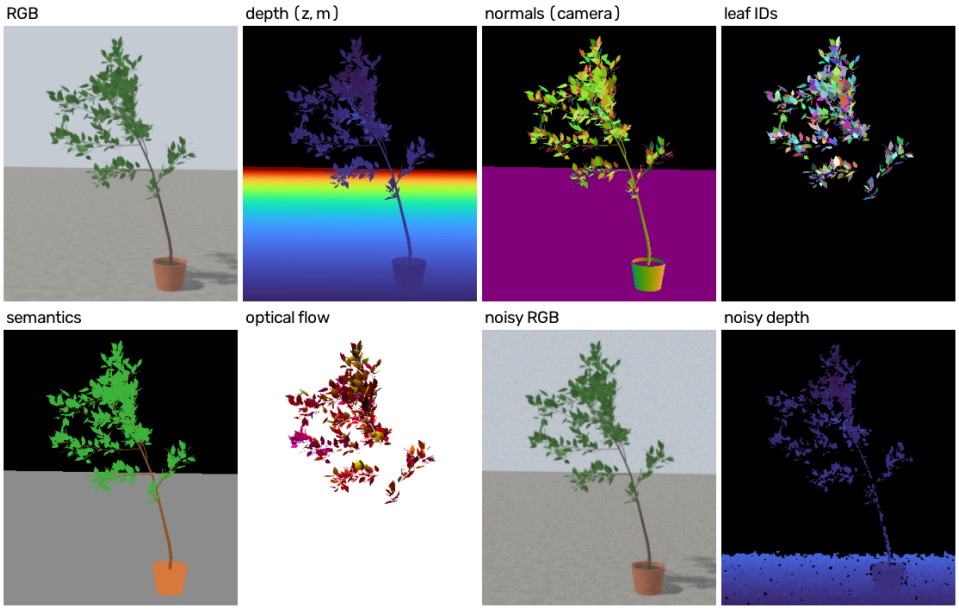


Figure 4: What is saved per frame (broad-leaved tree, 5 m/s): clean and noisy images and depth, plus ground-truth maps.

3.4. Image and depth noise

Every sequence is saved clean and with simulated noise. The noisy depth is lower in resolution and has errors that grow with distance, $\sigma(z) = a + bz^2$ (about 4 mm at 1 m by default); false "flying" points between leaves and background at depth edges; holes that are more likely near edges, on thin structures and on surfaces seen at a steep angle; and rounding to millimetres. The noisy colour image has small exposure changes, slight blur, pixel noise and JPEG compression. The noise is identical for all wind levels of a tree, so calm and windy runs differ only in the wind. The noise and wind parameters are configurable.

3.5. Checks

Every sequence is checked automatically (Table 3); all 40 of 40 pass. For example, floor points recovered from the depth lie within 3 μm of the true floor even in the image corners; using the wrong depth convention would move them by 147 mm.

Table 3: Automatic checks on all sequences.

Check (all 40 sequences)	Statistic	Value
World position re-projects onto its own pixel centre	max error	0.0036 px
Cross-frame re-projection of exported depth (static geometry)	worst median	0.002 px
Back-projected floor lies on $z = 0$ (image corners)	worst median	3 μm
same test if depth were ray distance	median	147 mm
16-bit PNG depth vs. float EXR depth	max difference	0.50 mm
Leaf-ID pixels not in table / inconsistent with semantics	total	0 px
Per-leaf transforms predict rendered flow (leaf pixels)	worst median	0.007 px
Rendered flow of static geometry	worst median	0.006 px

4. Experimental setup

Data. Each tree is rendered at four wind speeds, $U \in \{0, 1.5, 3, 5\}$ m/s, where 0 is calm, giving 40 sequences for 10 trees. In the main benchmark each sequence has 150 frames, one every 0.13 s (every fourth frame of a 30 fps video). Every eighth frame is held out for testing. We use trees 1, 3, 5 and 7, one per leaf type; low, medium and high wind mean 1.5, 3 and 5 m/s.

Methods. We test splatfacto (3DGS) and depth-nerfacto (a NeRF that also uses depth) from Nerfstudio (Tancik et al., 2023), given the true camera poses and started from the depth points of the same (clean or noisy) data. To test whether modelling motion helps, we also implemented our own static 3DGS and a deformable 3DGS on the same gsplat renderer, with the same start, loss (0.8 L1 + 0.2 D-SSIM), point management and 10,000 iterations. The deformable baseline is a re-implementation of the deformation-MLP idea of Yang et al. (2024) in our own renderer, not their official code: a small neural network (8 layers, 256 units, ten time bands) moves each Gaussian depending on its position and the time, and is trained for 8,000 iterations after a 2,000-iteration static warm-up. A third variant, the trimmed loss, ignores the 20 % of pixels with the largest error in each view, so that leaves the model cannot explain do not pull the geometry into a blur. Table 4 lists the training settings.

Measuring geometry. We compare reconstructions with the true leaf surface. Each reconstructed point is assigned to the nearest true surface (leaf, wood, pot or floor); points more than 3 cm from everything are counted as floaters. Accuracy-type measures use the Gaussian centres, so each Gaussian counts once. Coverage-type measures use points spread over each Gaussian's footprint, because the centres alone would make thin leaves look badly covered. We report the leaf F-score at 5 mm, a capped chamfer distance, the leaf-area

Table 4: Training settings. Each run used one NVIDIA T4 GPU (16 GB). Nerfstudio models use the defaults of version 1.1.5 except for the number of iterations.

	splatfacto	depth-nerfacto	our 3DGS (static, trimmed, deformable, per-cell)
Code	Nerfstudio 1.1.5, gsplat 1.4.0	Nerfstudio 1.1.5	gsplat 1.4.0 (Section 6: 1.5.3)
Iterations	7,000	15,000	10,000; motion after 2,000
Learning rates	positions $1.6 \cdot 10^{-4}$, decaying as for 30,000 steps; colour $2.5 \cdot 10^{-3}$ (higher SH /20); opacity 0.05; scale $5 \cdot 10^{-3}$; rotation 10^{-3}	fields and proposal networks 10^{-2}; 4,096 rays per batch; depth loss DS-NeRF, weight 10^{-3}, $\sigma = 0.01$	positions $1.6 \cdot 10^{-4}$ (constant, metres); colour $2.5 \cdot 10^{-3}$; opacity 0.05; scale $5 \cdot 10^{-3}$; rotation 10^{-3}; deformation network $8 \cdot 10^{-4}$, decaying to 2 %; per-cell motion $2 \cdot 10^{-5}$
Densification	every 100 steps from step 500; gradient threshold $8 \cdot 10^{-4}$; size threshold 0.01; opacity reset every 3,000 steps; cull opacity < 0.1	–	gsplat default strategy: every 100 steps from step 500; gradient threshold $2 \cdot 10^{-4}$; size threshold 0.01 m; opacity reset every 3,000 steps; prune opacity < 0.005
Stop splitting	step 15,000 (after training ends)	–	step 6,000
Max. Gaussians	no limit	–	no limit
Colour	SH degree 3, one degree more every 1,000 steps	–	RGB (SH degree 0)
Resolution	$\frac{1}{4}$ to step 3,000, $\frac{1}{2}$ to step 6,000, then 848×480	848×480	848×480
Pose refinement	off	on (SO3×R3, default)	off

error, the difference between leaf-angle distributions (Ross, 1981) and the median error of leaf centres. The chamfer distance is the average of two means over points, with each point-to-surface distance capped at 50 mm so that a few far floaters do not dominate: accuracy, from every Gaussian centre whose nearest true surface is a leaf (floaters included) to that surface, and completeness, from 300,000 points sampled on the true leaves to the nearest reconstructed leaf point. The leaf-centre error is the median over leaves. Tables average each measure over seeds and then over trees. Needles are thinner than a pixel, so for needle trees we treat each twig as one unit. Image quality is measured with PSNR on held-out views.

Rest pose or true pose. One can compare a reconstruction with the tree's rest pose (its shape in still air) or with the true position of every leaf at the moment of a test frame, which SynthTree also provides. We report both, calling the second *posed*; for dynamic methods the reconstruction is moved to that moment first. Posed scores are averaged over four held-out frames.

Splitting the error. For any error measure E (larger is worse) we define

$$E_{\text{method}} = E(\text{calm}, \text{clean}), \tag{5}$$

$$E_{\text{noise}} = E(\text{calm}, \text{noisy}) - E(\text{calm}, \text{clean}), \tag{6}$$

$$E_{\text{wind}}(U) = E(U, \text{noisy}) - E(\text{calm}, \text{noisy}), \tag{7}$$

$$E_{\text{int}}(U) = \big[E(U, \text{noisy}) - E(U, \text{clean})\big] - E_{\text{noise}}. \tag{8}$$

The last term shows whether noise and wind simply add up. We average over seeds and report the mean and spread over trees.

5. Results: what wind does to reconstructions

We ran 108 training runs for this part (Table 5): 60 for the benchmark and 48 for the comparison of static and dynamic models (Section 5). Splatfacto was trained on 4 trees under six conditions (noisy data at all four wind speeds, clean data calm and at 5 m/s) with 2 seeds each; depth-nerfacto, which takes 82 min per run, on two trees with one seed. The F-score differs by only about 0.014 between seeds, ten times less than the wind effects below.

Calm air. Even with exact camera poses and clean data, splatfacto reaches only F@5mm = 0.379 and a chamfer distance of 19.3 mm on leaves (Table 7), and it tilts leaves more steeply than they are. Thin leaves that hide each other are simply hard. Depth-nerfacto, which also learns from depth, does

Table 5: All training runs of the benchmark.

Study	Method	Trees	Wind U [m/s] per input (clean / noisy)	Seeds	Runs
Baseline study	deformable 3DGS (10-band)	1, 3, 5, 7	noisy: calm, 1.5, 3, 5	1	16
Baseline study	static 3DGS + trimmed loss	1, 3, 5, 7	noisy: calm, 1.5, 3, 5	1	16
Baseline study	static 3DGS (ours, gsplat)	1, 3, 5, 7	noisy: calm, 1.5, 3, 5	1	16
Benchmark	depth-nerfacto (Nerfstudio)	1, 7	noisy: calm, 1.5, 3, 5; clean: calm, 5	1	12
Benchmark	splatfacto (Nerfstudio)	1, 3, 5, 7	noisy: calm, 1.5, 3, 5; clean: calm, 5	2	48
Total reported training runs					108

Table 6: Benchmark results (splatfacto: 4 trees, 2 seeds; depth-nerfacto: 2 trees, 1 seed). F@5mm, Prec., Rec.: leaf F-score, precision and recall within 5 mm; CD: chamfer distance; Floaters: points more than 3 cm from any surface.

Method	Condition	F@5mm↑	Prec.↑	Rec.↑	CD [mm]↓	Centroid [mm]↓	Area err↓	Angle W1 [deg]↓	Floaters↓	PSNR [dB]↑
depth-nerfacto	calm / clean	0.558	0.75	0.47	6.4	5.6	0.53	10.1	0.01	35.5
	calm / noisy	0.525	0.66	0.45	6.7	6.5	0.55	10.9	0.01	26.2
	low / noisy	0.424	0.59	0.35	7.6	8.4	0.65	12.1	0.01	26.9
	medium / noisy	0.151	0.16	0.18	19.8	14.1	0.82	15.6	0.10	25.6
	high / noisy	0.178	0.22	0.15	17.0	18.1	0.85	12.5	0.04	26.2
	high / clean	0.193	0.30	0.15	14.6	14.5	0.85	12.3	0.04	29.2
splatfacto	calm / clean	0.379	0.38	0.42	19.3	6.7	0.58	12.6	0.19	36.1
	calm / noisy	0.439	0.39	0.60	17.8	8.6	0.40	12.0	0.32	29.1
	low / noisy	0.306	0.23	0.52	19.3	10.8	0.48	15.4	0.34	28.7
	medium / noisy	0.168	0.12	0.31	22.2	13.8	0.69	16.7	0.35	27.9
	high / noisy	0.099	0.07	0.19	25.9	20.6	0.81	17.7	0.43	27.1
	high / clean	0.098	0.08	0.14	26.0	19.4	0.86	16.7	0.19	29.5

clearly better in calm air (F@5mm = 0.558, chamfer distance 6.4 mm) and has almost no floaters.

Wind. Wind causes the largest damage. For splatfacto at 5 m/s the leaf F-score drops by 0.341, the chamfer distance rises by 8.1 mm and the error of leaf centres by 12.0 mm (Fig. 5). The F-score falls steadily with wind speed on all four trees, and for 7 of 8 tree–seed pairs. Depth-nerfacto loses about as much (F-score −0.348, chamfer distance +10.3 mm), so its better calm result does not protect it from moving leaves. Trees with broad, small and compound leaves suffer most; the needle tree, whose twigs move least (Fig. 3), suffers least. Per tree, the leaf motion at 5 m/s (Section 3.2), the absolute F-score drop and the relative drop are: tree 1: 48 mm, 0.48 (82 %); tree 3: 25 mm, 0.31 (85 %); tree 5: 47 mm, 0.44 (91 %); tree 7: 15 mm, 0.14 (42 %). Trees whose leaves move more lose more (Spearman $\rho = 1.00$; $\rho = 0.40$ for the relative drop), but with $n = 4$ trees this is a description, not a statistical test.

Noise. For splatfacto the noise term is negative (F-score -0.060, chamfer distance -1.5 mm): noisy data seem to *help*. The reason is that with noisy data splatfacto ends with 2.0× fewer but larger Gaussians near the tree (14,873 vs. 30,249). They cover more of the leaves (recall 0.42 → 0.60) without being more precise (precision 0.38 vs. 0.39), and floaters increase

Table 7: Error split into method, noise and wind (mean ± spread over trees). First row: absolute values in calm, clean conditions; other rows: increases, where positive means worse.

Method	Component	F@5mm drop	CD increase [mm]	Centroid increase [mm]
depth-nerfacto	Occlusion/method†	0.558 ± 0.078	6.4 ± 0.6	5.6 ± 1.5
	Noise	+0.033 ± 0.090	+0.2 ± 1.1	+1.0 ± 0.3
	Wind, $U = 1.5$ m/s	+0.101 ± 0.045	+1.0 ± 0.4	+1.8 ± 0.2
	Wind, $U = 3$ m/s	+0.374 ± 0.009	+13.1 ± 7.1	+7.6 ± 3.1
	Wind, $U = 5$ m/s	+0.348 ± 0.063	+10.3 ± 0.7	+11.6 ± 3.9
	Interaction, $U = 5$	-0.018 ± 0.089	+2.2 ± 3.1	+2.6 ± 4.5
splatfacto	Occlusion/method†	0.379 ± 0.117	19.3 ± 4.1	6.7 ± 5.7
	Noise	-0.060 ± 0.055	-1.5 ± 1.5	+1.9 ± 3.1
	Wind, $U = 1.5$ m/s	+0.134 ± 0.073	+1.5 ± 0.4	+2.2 ± 1.8
	Wind, $U = 3$ m/s	+0.272 ± 0.112	+4.3 ± 1.8	+5.2 ± 6.7
	Wind, $U = 5$ m/s	+0.341 ± 0.131	+8.1 ± 4.7	+12.0 ± 8.5
	Interaction, $U = 5$	+0.059 ± 0.054	+1.4 ± 1.5	-0.8 ± 2.1

(0.19 → 0.32). Noise therefore trades precision for coverage, and a single combined score can mistake this for an improvement, so precision and recall should be reported separately. For depth-nerfacto, which learns directly from the depth, noise hurts as expected (F-score 0.033). The size of the noise effect depends on our default noise settings; the trade-off between precision and coverage is the general lesson.

Do noise and wind add up?. At 5 m/s the interaction term for splatfacto is $+1.4 \pm 1.5$ mm in chamfer distance and -0.8 ± 2.1 mm in centre error, small compared with the wind terms of +8.1 and +12.0 mm (Table 7); for depth-nerfacto it is $+2.2 \pm 3.1$ mm. With four trees we see no sign that noise and wind interact strongly, but we cannot prove that they simply add up.

Images hide the damage. From calm to 5 m/s, PSNR on held-out views changes by -7 % while the leaf F-score changes by -75 % (Fig. 6). A model that blurs moving leaves into a plausible-looking average still produces good images. Only the per-leaf ground truth reveals the error.

Rest pose versus true pose. For static 3DGS (4 trees, noisy data), the F-score drop from calm to 5 m/s is 0.355 when compared with the rest pose but only 0.265 when compared with the true leaf positions (Table 8 and Fig. 7). At 5 m/s the reconstruction matches the moving leaves (F@5mm = 0.191) better than the rest pose (0.108), in 10 of 12 windy cases. A static model of a windy

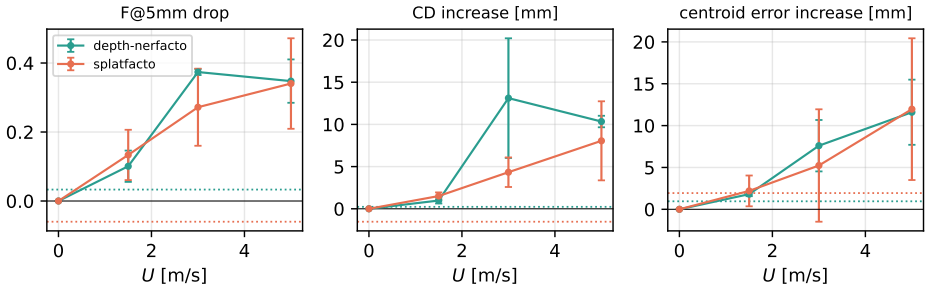


Figure 5: Error caused by wind at each wind speed (mean ± spread over trees). Dotted lines: error caused by noise.

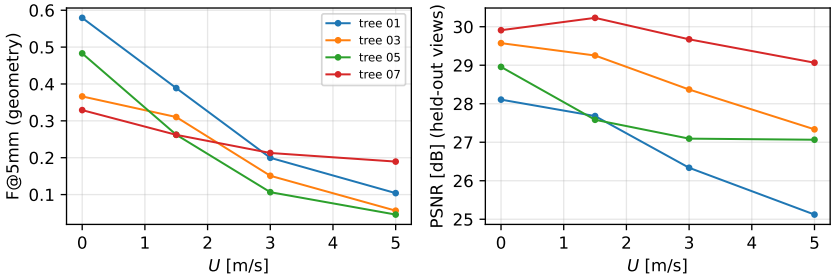


Figure 6: Leaf geometry (left) gets much worse with wind, image quality (right) hardly changes.

plant converges to its average, wind-bent shape, so comparing with the rest pose overstates the wind error by 34 %. This holds for our static 3DGS on four trees with one seed.

Deformable model. The deformable model does not recover the geometry lost to wind: at 5 m/s its posed F@5mm is 0.187, against 0.191 for the static model, and PSNR rises by only 0.1–0.3 dB (Table 8 and Fig. 7). Its learned motion shows why. The Gaussians move by a median of 0.9–4.0 mm, almost all by the same amount (95th percentile to median ratio 1.00), and no more in strong wind (2.4 mm) than in calm air (2.8 mm). The deformation has collapsed into a nearly uniform shift of the whole tree instead of moving individual leaves. We gave the network ten time frequencies instead of the usual six (Yang et al., 2024), because six only allow changes slower than about 0.8 Hz over a 20 s video; a first round with six behaved the same. Three things make the task hard: each moment is seen from only one viewpoint, frames are 0.13 s apart so flutter above 3.75 Hz is under-sampled, and thousands of small leaves move independently. Section 6 tests which of these matters.

Trimmed loss. Ignoring the worst-fitting pixels improves the posed F-score over plain static 3DGS in 16 of 16 tree–wind cases (one-sided sign test $p = 1.5 \times 10^{-5}$), raises precision (0.75 to 0.80 in calm air, 0.24 to 0.28 at 5 m/s) and recall (0.35 to 0.39, 0.17 to 0.19), lowers the chamfer distance by 1.5 mm at 5 m/s and reduces the share of floaters at 5 m/s from 0.13 to 0.10 (Table 8). But the gain is largest in calm air (+0.046) and smaller in strong wind (+0.027), and the drop from calm to 5 m/s (0.284) is as large as without it (0.265). It is a cheap, useful fix for the method error, not for the wind error: once leaves move, the bad pixels are no longer a small minority that can be ignored.

6. Can motion models recover the leaves?

The frames of the main benchmark are 0.13 s apart, and 83–87 % of the leaf motion happens faster than these frames can capture (above 3.75 Hz). The collapse of the deformable model could therefore be caused by the low frame rate rather than by the model. To separate the two, we rendered trees 1, 3, 5 and 7 again, calm and at 5 m/s, at the full 30 fps (600 frames, same trees, wind and camera path), and trained every model on all frames (30 fps) and on every fourth frame (7.5 fps, as in the benchmark).

Table 8: Static, trimmed-loss and deformable 3DGS (noisy data, 4 trees), compared with the rest pose and with the true leaf positions. Prec., Rec.: leaf precision and recall within 5 mm; Floaters: share of points more than 3 cm from any surface; Prec., Rec. and Floaters use the true leaf positions.

Wind	Method	F (rest)	F (posed)	Prec.	Rec.	CD rest [mm]	CD posed [mm]	Centroid posed [mm]	Floaters	PSNR [dB]
calm	static 3DGS	0.463	0.456	0.750	0.346	7.1	7.1	5.1	0.066	31.3
	static 3DGS + trimmed loss	0.499	0.502	0.801	0.386	6.3	6.3	4.7	0.063	31.2
	deformable 3DGS	0.396	0.434	0.755	0.321	7.5	7.2	5.1	0.069	31.5
low	static 3DGS	0.301	0.308	0.536	0.231	9.7	9.8	7.9	0.074	30.5
	static 3DGS + trimmed loss	0.347	0.357	0.589	0.273	8.5	8.5	7.7	0.068	30.4
	deformable 3DGS	0.275	0.295	0.540	0.216	10.2	10.0	7.7	0.078	30.6
medium	static 3DGS	0.166	0.217	0.361	0.168	14.0	12.8	12.0	0.094	29.6
	static 3DGS + trimmed loss	0.192	0.258	0.421	0.200	12.6	11.2	11.8	0.082	29.5
	deformable 3DGS	0.158	0.216	0.369	0.162	14.3	12.8	12.1	0.096	29.7
high	static 3DGS	0.108	0.191	0.239	0.171	18.8	15.1	15.7	0.127	28.7
	static 3DGS + trimmed loss	0.126	0.218	0.278	0.191	17.6	13.6	15.9	0.105	28.5
	deformable 3DGS	0.098	0.187	0.239	0.162	19.2	15.3	15.8	0.129	28.8

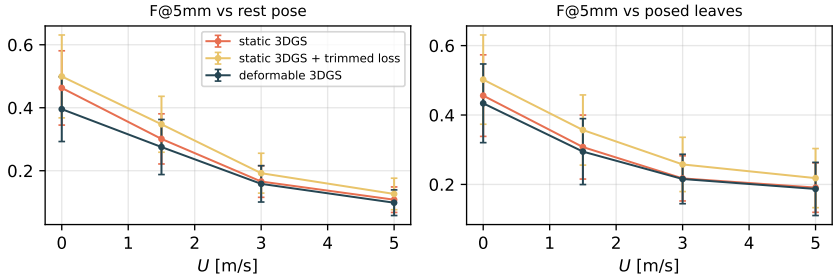


Figure 7: Leaf F-score against wind speed, compared with the rest pose (left) and the true leaf positions (right).

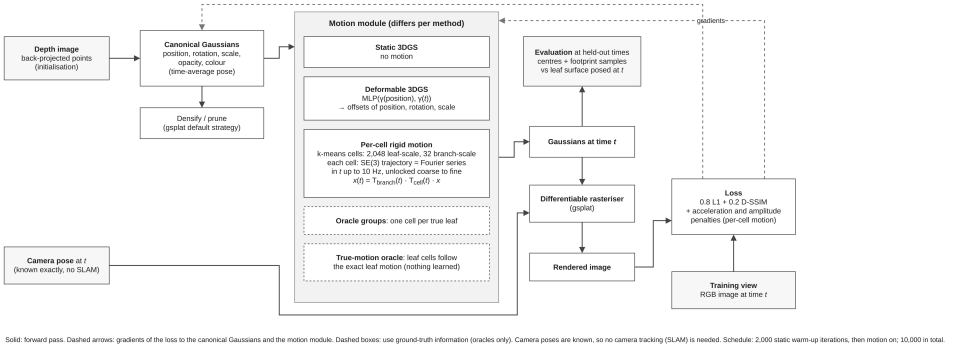


Figure 8: The reconstruction model. Gaussians are moved to the time of a training image by a motion module (the only part that differs between methods), rendered from the known camera pose and compared with the image. The two oracles (dashed) use ground truth.

Table 9: Motion models at 5 m/s (trees 1, 3, 5, 7). "Wind part of CD": increase in chamfer distance from calm to wind. Equal median and 95th-percentile offsets mean that all points moved together.

Method	fps	Cameras	Trees	F (posed)	CD posed [mm]	Wind part of CD [mm]	Centroid posed [mm]	PSNR [dB]	Offset med / p95 [mm]
static 3DGS	30	1	4	0.171	15.9	8.4	16.4	28.75	–
deformable 3DGS	30	1	4	0.160	16.5	8.5	17.1	28.75	3.9 / 3.9
per-cell rigid motion	30	1	4	0.154	15.3	6.8	15.1	29.25	2.7 / 10.9
per-cell rigid motion	7.5	1	4	0.166	14.8	6.6	14.4	27.87	2.8 / 9.6
per-leaf rigid motion (oracle groups)	30	1	4	0.179	14.5	6.3	15.4	29.27	2.8 / 11.5
true leaf motion (oracle)	30	1	4	0.468	7.8	0.0	5.8	30.89	3.2 / 25.3
static 3DGS	30	2	4	0.146	16.0	8.3	17.6	28.52	–
per-cell rigid motion	30	2	4	0.132	15.9	7.0	15.6	28.89	2.4 / 9.8
per-leaf rigid motion (oracle groups)	30	2	4	0.154	14.8	6.1	15.2	28.86	2.5 / 10.9
true leaf motion (oracle)	30	2	4	0.452	7.8	-0.2	5.5	30.50	2.9 / 24.8

6.1. A per-cell rigid-motion model

To test whether the collapse comes from the model's structure, we built a simple per-cell rigid-motion model, similar in spirit to superpoint and control-point Gaussians (Wan et al., 2024; Huang et al., 2024b) (Fig. 8). After the warm-up, the Gaussians are grouped by position into 2,048 small cells, about the size of a leaf, and 32 large cells, about the size of a branch. Each cell moves as a rigid body, $x \mapsto R_k(t)\,(x - p_k) + p_k + v_k(t)$, along its own path in time, described by a Fourier series up to 10 Hz whose higher frequencies are added gradually. A Gaussian follows its branch cell and its leaf cell. Branch cells are penalised ten times more for fast changes. Because every cell has its own parameters, the cells cannot merge into one global shift.

We also test two *oracles*, which use information a real recording does not have and therefore only show what would be possible. The first, the per-leaf oracle, groups the Gaussians by the true leaf identities (one cell per leaf). The second also gives each leaf its true motion, so that no leaf motion has to be learned.

All models use the same schedule as before (10,000 iterations, 2,000 without motion) and noisy data. At 5 m/s the static and per-cell models and the per-leaf oracle were trained with 3 seeds, whose F-scores differ by at most 0.016; other conditions use one seed (112 runs in total, Table 9). These runs use new renders, a newer version of the gsplat renderer (1.5.3 instead of 1.4.0) and newer software, which lowers all scores a little: on tree 1 the deformable model at 7.5 fps scores 0.449 (calm) and 0.137 (windy) against 0.501 and 0.162 in Table 8. Re-running static 3DGS with the old renderer version on the new data gives 0.493 and 0.150 (new version: 0.484 and 0.144; benchmark: 0.530 and 0.171), so the renderer version explains only a small part of the difference. We therefore only compare runs within Table 9.

Table 10: Complexity of the true leaf motion at 5 m/s (600 frames at 30 fps). PCs: principal components needed for 90 % of the variance of the leaf orientations or centres (at most three per leaf). Rigid R^2: share of the leaf-centre motion explained by one rigid transform of the whole tree, averaged over frames.

Tree	Leaf type	Leaves	PCs, orientations	PCs, centres	Rigid R^2
1	broad	624	264	97	0.53
3	small	535	244	5	0.72
5	compound	62	80	46	0.25
7	needle	1,672	275	124	0.59

6.2. The collapse is a modelling problem

At 30 fps the deformable model still learns a uniform shift: its offsets have a median of 3.9 mm and a 95th percentile of 3.9 mm (ratio 1.0 on every tree), and its F-score at 5 m/s (0.160) stays below that of static 3DGS (0.171). Four times more frames do not prevent the collapse. The true leaf motion explains why one smooth deformation cannot describe it. For each tree at 5 m/s we stack the leaf normals (orientations), or the leaf centres, of all 600 frames of the 30 fps sequence into a matrix with three rows per leaf and one column per frame, run PCA, and count the principal components needed for 90 % of the variance. Separately, we fit in every frame one rigid transform that maps the mean leaf-centre positions onto the current ones and report its R^2, the share of the leaf-centre motion it explains, averaged over frames. Orientations need 264, 244, 80 and 275 components on trees 1, 3, 5 and 7 (tree 5 has only 62 leaves), and a single rigid motion explains 0.53, 0.72, 0.25 and 0.59 of the centre motion (Table 10). Independently fluttering leaves do not fit a smooth or low-dimensional motion model, which is also what shared motion bases assume (Wang et al., 2024).

6.3. Per-cell motion removes the collapse but recovers little

The per-cell model learns different motion for different cells (offset median 2.7 mm, 95th percentile 10.9 mm; ratio 3.7–4.3). Compared with static 3DGS it reduces the wind part of the chamfer distance from 8.4 to 6.8 mm (−18 %), lowers the chamfer distance on 3 of 4 trees and raises PSNR by 0.5 dB. Its F-score at 5 m/s (0.154), however, is not higher than that of static 3DGS (0.171): it is clearly higher only on tree 1 and within the seed spread on trees 3 and 5. On the needle tree, where one cell contains many needles, it is much worse (0.171 vs. 0.261), and it adds small false motion in calm air (calm F 0.112 vs. 0.233 there), so part of its smaller wind term comes from a worse calm result. With the true leaf grouping the F-score

rises to 0.179 and the wind part of the chamfer distance falls by 24 %: even perfect grouping leaves about three quarters of the wind error.

Figures 9 and 10 show the per-cell model on tree 1 in every condition of the benchmark. The reconstructed images are hard to tell apart from the real ones at every wind speed, but in 3D the share of reconstructed points within 5 mm of a true leaf falls from 85 % in calm air to 25 % at 5 m/s with clean data (75 % to 24 % with noisy data), and the share of the true leaf surface it covers from 39 % to 15 % (43 % to 17 %).

6.4. A higher frame rate does not help

Trained on every fourth frame, the per-cell model reaches the same geometry (F 0.166, chamfer distance 14.8 mm) as with all frames (0.154, 15.3 mm). Without motion blur, the frame rate is not the limit: at the frame times, a slower trajectory fits the images as well as the true one, and each moment is still seen from only one viewpoint. The camera turns by only about 2° during one flutter cycle at 9 Hz, so for leaf flutter the orbit is almost a single fixed view (Gao et al., 2022).

6.5. A second camera does not help either

To test whether the single viewpoint is the limit, we added a second camera that follows the same circle 90° ahead, synchronised frame by frame, and trained on the images of both (testing still uses the first camera). The wind part of the chamfer distance stays the same for static 3DGS (8.4 vs. 8.3 mm with one and two cameras) and the per-cell model (6.8 vs. 7.0 mm), and falls only slightly with the true leaf grouping (6.3 vs. 6.1 mm); the drop in F-score is also unchanged (static 0.234 vs. 0.241, per-cell 0.207 vs. 0.193, per-leaf oracle 0.194 vs. 0.191). All scores are a little lower with two cameras, also in calm air (static calm F 0.405 vs. 0.386), probably because the same number of iterations is shared by twice as many images. In this setup, seeing each moment from a second direction is not the limit. Figures 11 and 12 compare the methods on tree 1.

6.6. The learned motion is far too small

What remains is estimating the leaf motion itself. The true leaf centres move by 15–48 mm from the rest pose at 5 m/s (Section 5), and by 9–31 mm (root mean square) around their time-averaged position, which is the motion a model must add to its average, wind-bent shape. But even with the true leaf grouping the learned motion has a median of only 2.8 mm and a 95th percentile of 11.5 mm. The models move the right Gaussians, but by far too little, with one or two cameras and at both frame rates.

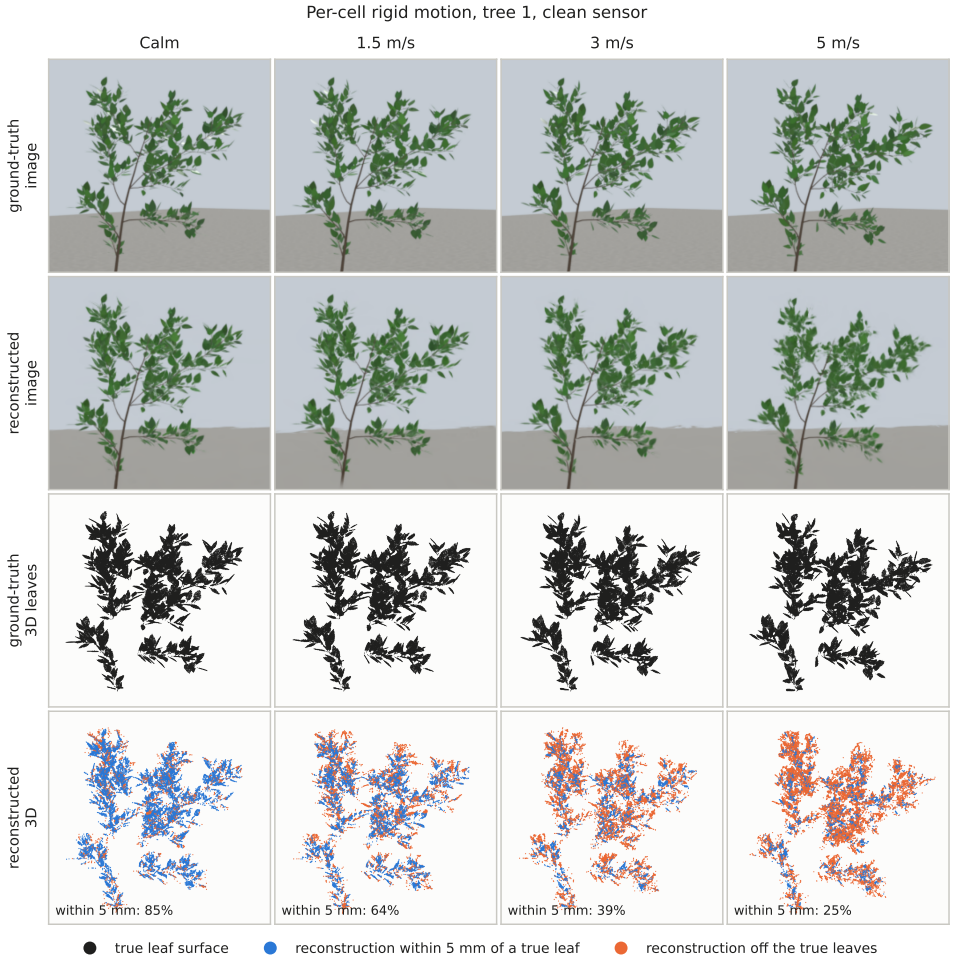


Figure 9: Per-cell model, tree 1, clean data, from calm to 5 m/s. The reconstructed images look like the real ones, but in 3D fewer and fewer points lie on the true leaves (blue: within 5 mm; orange: off).

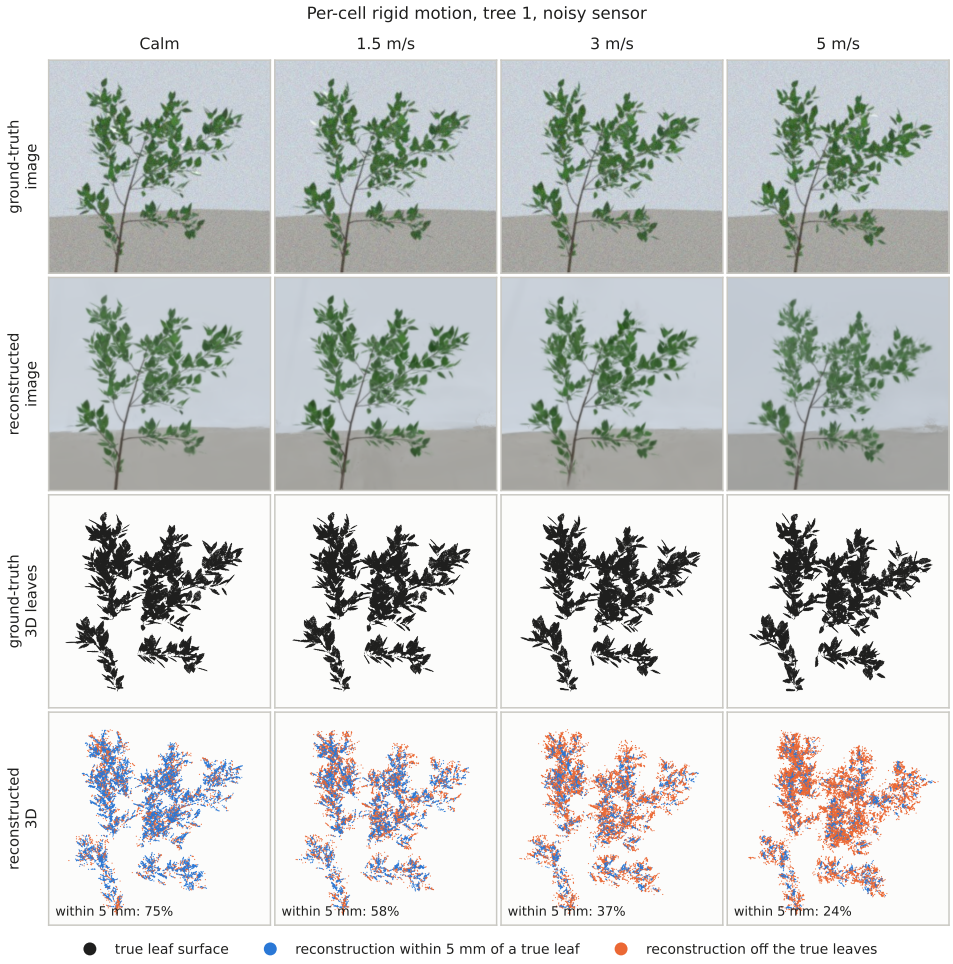


Figure 10: The same as Fig. 9 with noisy data.

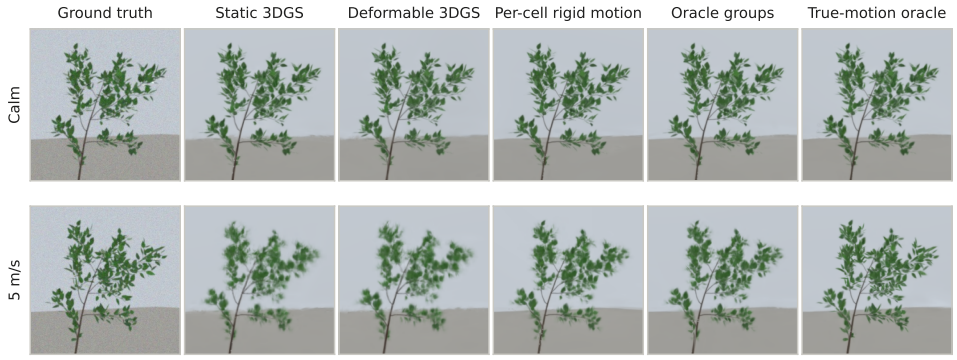


Figure 11: Tree 1, calm (top) and 5 m/s (bottom): the real image and each method's reconstruction. In wind, models that learn the motion blur the leaves; with the true motion they stay sharp.

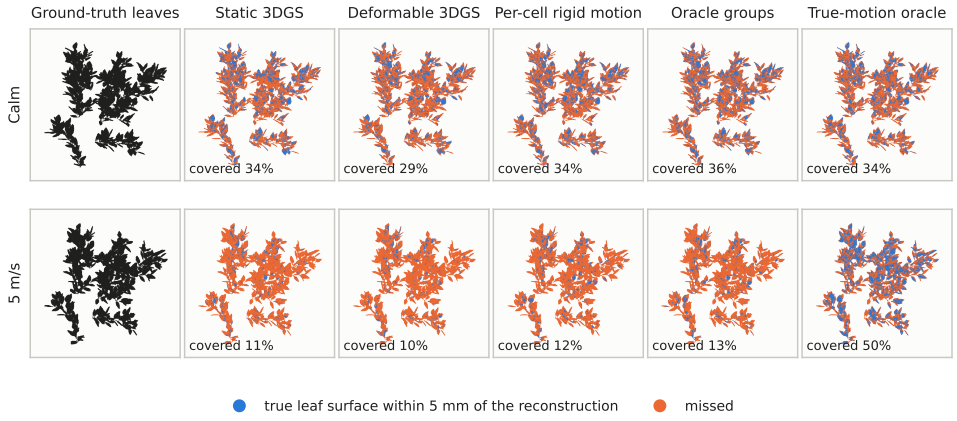


Figure 12: The same moment in 3D: the true leaf surface, blue where a method reconstructs it within 5 mm and orange where it misses it. In wind only the true-motion oracle recovers the leaves.

6.7. With the true motion, the wind error disappears

The second oracle settles the question. It is the same model with the same leaf grouping, but each leaf follows its true motion instead of a learned one. At 5 m/s it reaches a posed F-score of 0.468 (calm: 0.402), a chamfer distance of 7.8 mm and a leaf-centre error of 5.8 mm (calm: 5.3 mm), against 0.171, 15.9 mm and 16.4 mm for static 3DGS (Table 9). The wind part of the chamfer distance drops from 8.4 mm to 0.0 mm, and PSNR rises to 30.89 dB. On 3 of 4 trees the windy reconstruction is even better than the calm one, probably because moving leaves show more of their surface to the camera over time; only the needle tree, where single needles are hard to group, gains little (F 0.220). Two cameras give the same result (F 0.452, wind part -0.2 mm). In Figs. 11 and 12 it is the only model whose windy leaves stay sharp and on the true surface. The representation is therefore good enough: once the motion is known, Gaussians reconstruct leaves in wind as well as in still air. This agrees with Chen and Lou (2026), who find that learned tree motion does not match the true motion, and adds that the problem lies in estimating the motion, not in representing it. Estimating large, fast, independent leaf motion from images is the remaining problem.

7. Discussion and limitations

All results come from simulation. The noise and wind settings are plausible defaults, not values measured for a particular plant and fan, so the absolute errors on real recordings may differ. The comparisons that the paired design is built for (calm against windy, clean against noisy, one method against another) do not depend on these exact values. Checking the trends on real recordings is the most important next step. A practical design is to reconstruct from a moving camera and to evaluate with synchronised static cameras, as in the iPhone dataset of Gao et al. (2022), triangulating leaf positions as in multi-camera studies of trees in wind (Kammel and Reiterer, 2024).

The generator trades physical realism for exact ground truth: branches bend in rigid segments, compound leaves move as one piece, leaves can pass through each other and gusts are the same everywhere. Motion blur and rolling shutter, which a real handheld recording of moving leaves would add, were switched off, so the wind errors reported here are probably lower than they would be in practice.

Our conclusions apply to the methods we tested: splatfacto and depth-nerfacto from Nerfstudio and our static, trimmed-loss, deformable and per-cell 3DGS. We did not test 2DGS (Huang et al., 2024a), other NeRF variants

or dedicated dynamic methods (Wu et al., 2024; Luiten et al., 2024), and with four trees and one to three seeds the per-tree results are descriptive. The deformable baseline is our re-implementation of Yang et al. (2024); we have not checked that their official code collapses in the same way on our data, nor that our version learns non-rigid motion on a standard dynamic-scene benchmark. The per-cell model is a simple test of motion structure, not a new method; closely related models exist (Wan et al., 2024; Huang et al., 2024b; Wang et al., 2024). The motion-model study uses new renders and newer software, so its absolute numbers are not directly comparable with the benchmark; its calm and 7.5 fps runs use one seed. The second-camera test used two cameras 90° apart on one circle and a fixed number of iterations. More or more widely spaced cameras, longer training, motion blur, and extra guidance for large leaf motion, for example from the optical flow that SynthTree saves, are natural next steps, and the saved per-leaf motion allows them to be evaluated exactly.

8. Conclusion

SynthTree provides paired calm and windy RGB-D videos of plants with exact per-leaf ground truth and a way to split reconstruction error into method, noise and wind. In simulation, across four leaf types, wind is the main source of error for neural plant reconstruction; image metrics hide most of it, comparing with the rest pose overstates it, and neither our deformable baseline nor a robust loss recovers it. The deformable baseline fails because its motion collapses into a uniform shift, and this happens at 30 fps as well, so the model, not the frame rate, is to blame. Per-cell motion removes the collapse, but even with the true leaf grouping it recovers only about a quarter of the wind error, and neither more frames nor a second camera help. The learned leaf motion stays far smaller than the true motion, and when the true motion is given, the wind error disappears. Recovering fast leaf motion from images is therefore the key problem to solve next, and the trends should be checked on real recordings.

Data and code availability

Code for the generator, training and evaluation (MIT licence): `https://github.com/bibekgyawali2/synthtree`. Per-run results of all 108 benchmark runs are included as CSV files, and `scripts/paper/make_assets.py` recreates every table, figure and number of the benchmark from them; the training scripts are under `kaggle/`. The 112 runs of the motion-model study

are included as JSON files and `scripts/paper/make_stage1_assets.py` recreates their table and numbers; their renders are produced by `configs/paper2_stage1.yaml` and `configs/paper2_stage1_cam2.yaml`. Dataset (40 sequences, 9.2 GB, CC BY 4.0): `https://www.kaggle.com/datasets/bibekgyawali2/synthtree-v1`, available to editors and reviewers on request and to be made public with a DOI on publication; it can also be regenerated exactly from the released configuration files.

CRediT authorship contribution statement

Bibek Gyawali: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare no competing interests.

References

Chen, W., Lou, E., 2026. Wind on trees: Testing physical grounding in dynamic 4D Gaussian splatting. arXiv preprint arXiv:2609.17810 .

Denninger, M., Winkelbauer, D., Sundermeyer, M., Boerdijk, W., Knauer, M., Strobl, K.H., Humt, M., Triebel, R., 2023. BlenderProc2: A procedural pipeline for photorealistic rendering. Journal of Open Source Software 8, 4901. doi:10.21105/joss.04901.

Gao, H., Li, R., Tulsiani, S., Russell, B., Kanazawa, A., 2022. Monocular dynamic view synthesis: A reality check, in: Advances in Neural Information Processing Systems (NeurIPS).

Handa, A., Whelan, T., McDonald, J., Davison, A.J., 2014. A benchmark for RGB-D visual odometry, 3D reconstruction and SLAM, in: IEEE International Conference on Robotics and Automation (ICRA), pp. 1524–1531. doi:10.1109/icra.2014.6907054.

Huang, B., Yu, Z., Chen, A., Geiger, A., Gao, S., 2024a. 2D gaussian splatting for geometrically accurate radiance fields, in: ACM SIGGRAPH 2024 Conference Papers, pp. 1–11. doi:10.1145/3641519.3657428.

Huang, Y.H., Sun, Y.T., Yang, Z., Lyu, X., Cao, Y.P., Qi, X., 2024b. SC-GS: Sparse-controlled Gaussian splatting for editable dynamic scenes, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

Jensen, R., Dahl, A., Vogiatzis, G., Tola, E., Aanæs, H., 2014. Large scale multi-view stereopsis evaluation, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 406–413. doi:10.1109/cvpr.2014.59.

Kammel, F.O., Reiterer, A., 2024. Conceptualization and first realization steps for a multi-camera system to capture tree streamlining in wind. Forests 15, 1846. doi:10.3390/f15111846.

Kerbl, B., Kopanas, G., Leimkühler, T., Drettakis, G., 2023. 3D gaussian splatting for real-time radiance field rendering. ACM Transactions on Graphics 42, 1–14. doi:10.1145/3592433.

Knapitsch, A., Park, J., Zhou, Q.Y., Koltun, V., 2017. Tanks and temples: Benchmarking large-scale scene reconstruction. ACM Transactions on Graphics 36, 1–13. doi:10.1145/3072959.3073599.

de Langre, E., 2008. Effects of wind on plants. Annual Review of Fluid Mechanics 40, 141–168. doi:10.1146/annurev.fluid.40.111406.102135.

Li, Y., Fan, X., Mitra, N.J., Chamovitz, D., Cohen-Or, D., Chen, B., 2013. Analyzing growing plants from 4D point cloud data. ACM Transactions on Graphics 32, 1–10. doi:10.1145/2508363.2508368.

Lin, Y., Dai, Z., Zhu, S., Yao, Y., 2024. Gaussian-Flow: 4D reconstruction with dynamic 3D Gaussian particle, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

Luiten, J., Kopanas, G., Leibe, B., Ramanan, D., 2024. Dynamic 3D gaussians: Tracking by persistent dynamic view synthesis, in: International Conference on 3D Vision (3DV), pp. 800–809. doi:10.1109/3dv62453.2024.00044.

Martin-Brualla, R., Radwan, N., Sajjadi, M.S.M., Barron, J.T., Dosovitskiy, A., Duckworth, D., 2021. NeRF in the wild: Neural radiance fields for unconstrained photo collections, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7206–7215. doi:10.1109/cvpr46437.2021.00713.

Mildenhall, B., Srinivasan, P.P., Tancik, M., Barron, J.T., Ramamoorthi, R., Ng, R., 2020. NeRF: Representing scenes as neural radiance fields for view synthesis, in: European Conference on Computer Vision (ECCV), pp. 405–421. doi:10.1007/978-3-030-58452-8_24.

Müller, T., Evans, A., Schied, C., Keller, A., 2022. Instant neural graphics primitives with a multiresolution hash encoding. ACM Transactions on Graphics 41, 1–15. doi:10.1145/3528223.3530127.

Paulus, S., 2019. Measuring crops in 3D: Using geometry for plant phenotyping. Plant Methods 15. doi:10.1186/s13007-019-0490-0.

Prusinkiewicz, P., Lindenmayer, A., 1990. The Algorithmic Beauty of Plants. Springer. doi:10.1007/978-1-4613-8476-2.

Pumarola, A., Corona, E., Pons-Moll, G., Moreno-Noguer, F., 2021. D-NeRF: Neural radiance fields for dynamic scenes, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10313–10322. doi:10.1109/cvpr46437.2021.01018.

Rochow, A., Marcic, J., Seliunina, S., Behnke, S., 2025. Iterative motion compensation for canonical 3D reconstruction from UAV plant images captured in windy conditions. arXiv preprint arXiv:2510.15491 .

Ross, J., 1981. The Radiation Regime and Architecture of Plant Stands. Dr W. Junk Publishers. doi:10.1007/978-94-009-8647-3.

Runions, A., Lane, B., Prusinkiewicz, P., 2007. Modeling trees with a space colonization algorithm, in: Eurographics Workshop on Natural Phenomena, pp. 63–70.

Sabour, S., Vora, S., Duckworth, D., Krasin, I., Fleet, D.J., Tagliasacchi, A., 2023. RobustNeRF: Ignoring distractors with robust losses, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 20626–20636. doi:10.1109/cvpr52729.2023.01976.

Schunck, D., Magistri, F., Rosu, R.A., Cornelißen, A., Chebrolu, N., Paulus, S., Léon, J., Behnke, S., Stachniss, C., Kuhlmann, H., Klingbeil, L., 2021. Pheno4D: A spatio-temporal dataset of maize and tomato plant point clouds for phenotyping and advanced plant analysis. PLOS ONE 16, e0256340. doi:10.1371/journal.pone.0256340.

Sturm, J., Engelhard, N., Endres, F., Burgard, W., Cremers, D., 2012. A benchmark for the evaluation of RGB-D SLAM systems, in: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 573–580. doi:10.1109/iros.2012.6385773.

Tadrist, L., Saudreau, M., Hémon, P., Amandolese, X., Marquier, A., Leclercq, T., de Langre, E., 2018. Foliage motion under wind, from leaf flutter to branch buffeting. Journal of the Royal Society Interface 15, 20180010. doi:10.1098/rsif.2018.0010.

Tancik, M., Weber, E., Ng, E., Li, R., Yi, B., Kerr, J., Wang, T., Kristoffersen, A., Austin, J., Salahi, K., Ahuja, A., McAllister, D., Kanazawa, A., 2023. Nerfstudio: A modular framework for neural radiance field development, in: ACM SIGGRAPH 2023 Conference Proceedings, pp. 1–12. doi:10.1145/3588432.3591516.

Tzathas, P., Hu, J., Meuleman, A., Cordonnier, G., Drettakis, G., 2026. Adaptive spatio-temporal 3D Gaussian splatting for scenes with oscillatory motion. Computer Graphics Forum 45.

Wan, D., Lu, R., Zeng, G., 2024. Superpoint Gaussian splatting for real-time high-fidelity dynamic scene reconstruction, in: Proceedings of the 41st International Conference on Machine Learning (ICML), pp. 49957–49972.

Wang, Q., Ye, V., Gao, H., Austin, J., Li, Z., Kanazawa, A., 2024. Shape of motion: 4D reconstruction from a single video. arXiv preprint arXiv:2407.13764 .

Weber, J., Penn, J., 1995. Creation and rendering of realistic trees, in: Proceedings of SIGGRAPH '95, pp. 119–128. doi:10.1145/218380.218427.

Wu, G., Yi, T., Fang, J., Xie, L., Zhang, X., Wei, W., Liu, W., Tian, Q., Wang, X., 2024. 4D gaussian splatting for real-time dynamic scene rendering, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 20310–20320. doi:10.1109/cvpr52733.2024.01920.

Yang, Z., Gao, X., Zhou, W., Jiao, S., Zhang, Y., Jin, X., 2024. Deformable 3D gaussians for high-fidelity monocular dynamic scene reconstruction, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 20331–20341. doi:10.1109/cvpr52733.2024.01922.